

Лекція 3.1.

Тема: Введення в PHP. Вираження і управління процесом виконання програми у PHP.

План лекції

1. Можливості PHP.
2. Встановлення і налаштування ПЗ.
3. Перша PHP-програма.
4. Основний синтаксис
5. Змінні, константи і оператори.
6. Типи даних.

Зміст лекції

1. Можливості PHP.

"PHP може все", - заявляють його творці. В першу чергу PHP використовується для створення скриптів, що працюють на стороні сервера, для цього його, власне, і придумали. PHP здатний вирішувати ті ж завдання, що і будь-які інші CGI - скрипти, в тому числі обробляти дані html-форм, динамічне генерування html сторінки і т.п. Але є й інші області, де може використовуватися PHP. Всього виділяють три основні області застосування PHP.

- Перша область, як вже говорилося, - це створення додатків (скриптів), які виконуються на стороні сервера. PHP найбільш широко використовується саме для створення такого роду скриптів. Для того щоб працювати таким чином, знадобиться PHP - парсер (тобто обробник php - скриптів) і web-сервер для обробки скрипта, браузер для перегляду результатів роботи скрипта, ну, і, звичайно, будь-якої текстовий редактор для написання самого php-коду. Парсер PHP поширюється у вигляді CGI - програми або серверного модуля.

- Друга область - це створення скриптів, що виконуються в командному рядку. Тобто за допомогою PHP можна створювати такі скрипти, які будуть виконуватися, незалежно від web-сервера і браузера, на конкретній машині. Для такої роботи потрібно лише парсер PHP (в цьому випадку його називають інтерпретатором командного рядка (cli, command line interpreter)). Цей спосіб роботи підходить, наприклад, для скриптів, які повинні виконуватися регулярно за допомогою різних планувальників завдань або для вирішення завдань простий обробки тексту.

- І остання область - це створення GUI-додатків (графічних інтерфейсів), що виконуються на стороні клієнта. В принципі це не найкращий спосіб використовувати PHP, особливо для початківців, але якщо ви вже досконально вивчили PHP, то такі можливості мови можуть виявитися вельми корисні. Для застосування PHP в цій області буде потрібно спеціальний інструмент - PHP-GTK, який є розширенням PHP.

Отже, сфера застосування PHP достатньо обширна і різноманітна. Проте існує безліч інших мов програмування, здатних вирішувати схожі завдання. Чому варто вивчати PHP? Що це нам дає? По-перше, PHP дуже простий у вивченні. Достатньо ознайомитися лише з основними правилами синтаксису і принципами його роботи, і можна починати писати власні програми, причому братися за такі завдання, вирішення яких на іншій мові вимагало б серйозної підготовки.

По-друге, PHP підтримується майже на всіх відомих платформах, майже у всіх операційних системах і на самих різних серверах. Це теж дуже важливо. Навряд чи комусь захочеться переходити, наприклад, від роботи під Windows до роботи під Linux або від сервера IIS до сервера Apache тільки для того, щоб вивчити ще одну мову програмування.

У PHP поєднуються дві найпопулярніші парадигми програмування - об'єктна і процедурна. В PHP4 більш повно підтримується процедурне програмування, але є можливість писати програми і в об'єктному стилі.

Якщо говорити про можливості сьогоденного PHP, то вони виходять далеко за рамки тих, що були реалізовані в його перших версіях. За допомогою PHP можна створювати зображення, PDF-файли, флеш-ролики, в нього включена підтримка великого числа сучасних баз даних, вбудовані функції для роботи з текстовими даними будь-яких форматів, включаючи XML, і функції для роботи з файловою системою. PHP підтримує взаємодію з різними сервісами за допомогою відповідних протоколів, таких як протокол управління доступом до теки LDAP, протокол роботи з мережним устаткуванням SNMP, протоколи передачі повідомлень IMAP, NNTP і POP3, протокол передачі гіпертексту HTTP і т.д.

Звертаючи увагу на взаємодію між різними мовами, слід згадати про підтримку об'єктів Java і можливості їх використання в якості об'єктів PHP. Для доступу до віддалених об'єктів можна використовувати розширення CORBA.

Для роботи з текстовою інформацією PHP успадкував (з невеликими змінами) механізми роботи з регулярними виразами з мови Perl і UNIX-систем. Для обробки XML - документів можна використовувати як стандарти DOM і SAX, так і API для XSLT - трансформація.

2. Встановлення і налаштування ПО.

Можливості мови ми обговорили, області застосування розглянули, історію вивчили. Тепер можна приступати до установки необхідного інструментарію. Оскільки в якості практичної основи курсу ми будемо розглядати завдання, які вирішуються за допомогою технології клієнт-сервер, і PHP відповідно використовуватиметься для створення скриптів, оброблюваних сервером, нам потрібно встановити web-сервер і інтерпретатор PHP. Як web-сервера виберемо, наприклад, Apache, як найбільш популярний серед web-розробників. Для перегляду результатів роботи програм нам знадобиться web-браузер.

Для швидкого встановлення всіх потрібних програм є дуже багато комплексів які це автоматизують, достатньо в пушуку набрати XAMP. Також можна подивитися тут: <https://ru.wikipedia.org/wiki/WAMP>

Я пропоную використовувати **XAMP** для лабораторних робіт буде достатньо версії Basic. Автор проекту розповсюджує цей комплекс безкоштовно.

3. Перша PHP-програма.

Розглянемо приклад.

```
<Html>
<Head>
<Title> Приклад </title>
</Head>
<Body>
<?Php
echo "<p> Привіт, я - скрипт PHP! </p>";
?>
</Body>
</Html>
```

Приклад 3.1.1. Простий html-файл з вбудованим кодом на PHP

Це простий HTML-файл, в який вбудований за допомогою спеціальних тегів код, написаний на мові PHP.

Як ми вже відзначали вище, PHP схожий на C і Perl. Однак наведена тут програма сильно відрізняється від аналогічних за змістом програм на мовах програмування C і Perl.

Тут не потрібно писати купу спеціальних команд для виведення HTML. Пишеться безпосередньо код на HTML, в який можна вбудовувати PHP-код, який здійснює будь-які дії (наприклад, виводить текст на екран, як в нашому прикладі). Недоліком PHP в порівнянні з C і Perl, є недостатня швидкість виконання складних скриптів.

PHP-скрипти - це програми, які виконуються і обробляються сервером. Так що порівнювати його з скриптовими мовами типу JavaScript неможливо, тому що написані на них скрипти виконуються на машині клієнта. У чому відмінність скриптів, виконуваних на сервері і на клієнті? Якщо скрипт обробляється сервером, клієнту посилаються тільки результати роботи скрипта. Наприклад, якщо на сервері виконувався скрипт, подібний до наведеного вище, клієнт отримає згенеровану HTML-сторінку вигляду:

```
<Html>
  <Head>
    <Title> Приклад </title>
  </Head>
  <Body>
    <P> Привіт, я - скрипт PHP! </ P>
  </Body>
</Html>
```

У цьому випадку клієнт не знає, який код виконується. Можна навіть налаштувати свій сервер таким чином, щоб HTML-файли оброблялися процесором PHP, так що клієнти навіть не зможуть дізнатися, чи отримують вони звичайний HTML-файл або результат виконання скрипта. Якщо ж скрипт обробляється клієнтом (наприклад, це програма на мові JavaScript), то клієнт отримує сторінку, яка містить код скрипта.

Ми вже згадували вище, що PHP - скрипти вбудовуються в HTML-код. Виникає питання, яким чином? Є кілька способів. Один з них наведено в найпершому прикладі - за допомогою відкриваючого тега `<?Php` і закриваючого тегу `?>`. Такого виду спеціальні теги дозволяють перемикатися між режимами HTML і PHP. Цей синтаксис найбільш переважний, оскільки дозволяє задіяти PHP в XML-сумісних програмах (наприклад, написаних на мові XHTML), але тим не менш можна використовувати такі альтернативні варіанти (команда `echo "Some text"`; виводить на екран текст "Some text".):

```
<? echo "Це найпростіша інструкція для обробки PHP ";?>
<Script language = "php">
  echo "Деякі редактори (FrontPage) віддають перевагу робити так ";
</Script>
<% Echo "Можна використовувати теги в стилі ASP ";%>
```

Перший з цих способів не завжди доступний. Щоб їм користуватися, потрібно включити короткі теги або за допомогою функції `short_tags()` для PHP 3, або включивши установку `short_open_tag` в конфігураційному файлі PHP, або скомпілювавши PHP з параметром `--enable-short-tags`. Навіть якщо це включено за замовчуванням в `php.ini-dist`, Використання коротких тегів не рекомендується. Другий спосіб аналогічний вставці, наприклад, JavaScript-коду і використовує для цього відповідний html тег. Тому використовувати його можна завжди, але це робиться рідко через його громіздкість. Третій спосіб можна застосувати, тільки якщо теги в стилі ASP були включені, використовуючи конфігураційну установку `asp_tags`.

Коли PHP обробляє файл, він просто передає його текст, поки не зустріне один з перерахованих спеціальних тегів, який повідомляє йому про необхідність почати інтерпретацію тексту як коду PHP. Потім він виконує весь знайдений код до закриваючого тега, що говорить інтерпретатору, що далі знову йде просто текст. Цей механізм дозволяє впроваджувати PHP-код в HTML - все за межами тегів PHP залишається незмінним, тоді як всередині інтерпретується як код. Зауважимо також, що php-файл не схожий на CGI-скрипт. Php файл не повинен бути виконуваним або ще якимось чином поміченим.

Для того щоб відправити php-файл на обробку серверу, потрібно в рядку браузера набрати шлях до цього файлу на сервері. Скрипти php повинні розташовуватися там, де дозволений доступ через www, наприклад там же, де лежить домашня сторінка. Якщо php-файл лежить на локальній машині, то його можна обробити за допомогою інтерпретатора командного рядка.

4. Основний синтаксис

Перше, що потрібно знати щодо синтаксису PHP, - це те, як він вбудовується в HTML-код, як інтерпретатор довідається, що це код на мові PHP. У попередній лекції ми вже говорили про це. Повторюватися не будемо, зазначимо лише, що в прикладах ми найчастіше будемо використовувати варіант `<?Php?>`, і іноді скорочений варіант `<? ?>`.

Поділ інструкцій

Програма на PHP (та й будь-якою іншою мовою програмування) - це набір команд (інструкцій). Оброблювачу програми (парсеру) необхідно якось відрізнити одну команду від іншої. Для цього використовуються спеціальні символи - роздільники. У PHP інструкції розділяються так само, як і в Cі або Perl, - кожен вираз закінчується крапкою з комою.

Закриває тег `"?>"` Також має на увазі кінець інструкції, тому перед ним крапку з комою не ставлять. Наприклад, наступні фрагменти коду еквівалентні:

```
<?Php
echo "Hello, world!"; // крапка з комою в кінці команди обов'язкова
?>
<?Php echo "Hello, world!" ?>
<!-- крапка з комою опускається через "?>" -->
```

Коментарії.

Часто при написанні програм виникає необхідність робити які-небудь коментарі до коду, які ніяк не впливають на сам код, а тільки пояснюють його. Це важливо при створенні великих програм і в разі, якщо кілька людей працюють над однією програмою. При наявності коментарів в програмі в її коді розібратися набагато простіше. Крім того, якщо вирішувати задачу по частинах, недороблені частини рішення також зручно коментувати, щоб не забути про них в подальшому. В усіх мовах програмування передбачена можливість включати коментарі в код програми. PHP підтримує кілька видів коментарів: в стилі Cі, C++ і оболонки Unix. Символи `//` і `#` позначають початок однорядкових коментарів, `/*` і `*/` - відповідно початок і кінець багато рядкових коментарів.

```
<?Php
echo "Мене звуть Вася";
// Це однорядковий коментар
// в стилі C++
echo "Прізвище моє Петров";
/* Це багаторядковий коментар.
Тут можна написати кілька рядків.
При виконанні програми все, що
знаходиться тут (закоментувавши),
буде проігноровано. */
echo "Я вивчаю PHP";
# Це коментар в стилі
# Оболонки Unix
?>
```

Приклад 3.1.1. Використання коментарів в PHP

5. Змінні, константи і оператори.

Важливим елементом кожної мови є змінні, константи і оператори, що застосовуються до цих змінним і константам. Розглянемо, як виділяються і обробляються ці елементи в PHP.

Змінні.

Змінна у PHP позначається знаком долара, за яким слідує її ім'я. наприклад:

`$my_var`

Ім'я змінної чутливо до регістру, тобто змінні `$my_var` і `$My_var` різні.

Імена змінних відповідають тим же правилам, що і інші назви в PHP: правильне ім'я змінної повинно починатися з букви або символу підкреслення з подальшими в будь-якій кількості буквами, цифрами або символами підкреслення.

У PHP з змінні завжди привласнювалися по значенню. Тобто коли ви привласнюєте вираз змінної, всі значення оригінального виразу копіюються в цю змінну. Це означає, наприклад, що після присвоєння одній змінній значення іншої, зміна однієї з них не впливає на значення іншої.

```
<?Php
```

```
$First = 'Text'; // Надаємо $first значення 'Text'
```

```
$Second = $first; // Надаємо $second значення змінної $first
```

```
$First = 'New text'; // Змінюємо значення $first на 'New text'
```

```
echo "Змінна з ім'ям first дорівнює". $first <br> "; // виводимо значення $first
```

```
echo "Змінна з ім'ям second дорівнює". $second "; // виводимо значення $second
```

```
?>
```

Приклад 3.1.2. Присвоєння за значенням

Результат роботи цього скрипта буде наступним:

Змінна з ім'ям first дорівнює New text

Змінна з ім'ям second дорівнює Text

PHP 4, крім цього, пропонує ще один спосіб привласнення значень змінним: привласнення по посиланню. Для того, щоб привласнити значення змінної по посиланню, це значення повинне мати ім'я, тобто воно повинно бути представлено будь-якої змінної. Щоб вказати, що значення однієї змінної присвоюється іншій змінної за посиланням, потрібно перед ім'ям першої змінної поставити знак амперсанд `&`.

Розглянемо той же приклад, що і вище, тільки будемо присвоювати значення змінної `first` змінної `second` за посиланням:

```
<?Php
```

```
$First = 'Text'; // Надаємо $first значення 'Text'
```

```
$Second = &$first;
```

```
/* Робимо посилання на $first через $Second. Тепер значення цих змінних  
будуть завжди збігатися */
```

```
// Змінимо значення $first на 'New text'
```

```
$first = 'New text';
```

```
echo "Змінна з ім'ям first"."Дорівнює $first <br>";
```

```
// виведемо значення обох змінних
```

```
echo "Змінна з ім'ям second"."Дорівнює $Second";
```

```
?>
```

Приклад 3.1.3. Присвоєння по посиланню

Цей скрипт виведе наступне:

Змінна з ім'ям first дорівнює New text.

Змінна з ім'ям second дорівнює New text.

Тобто разом з змінною `$first` змінилася і змінна `$Second`.

Константи.

Для зберігання постійних величин, тобто таких величин, значення яких не змінюється в ході виконання скрипта, використовуються константи. Такими величинами

можуть бути математичні константи, паролі, шляхи до файлів і т.п. Основна відмінність константи від змінної полягає в тому, що їй не можна привласнити значення більше одного разу і її значення не можна анулювати після її оголошення. Крім того, у константи немає приставки у вигляді знака долара і її не можна визначити простим привласненням значення. Як же тоді можна визначити константу? Для цього існує спеціальна функція **define()**. Її синтаксис такий:

```
define("Ім'я_константи", "Значення_константи", [Нечутливість_до_регістру])
```

За замовчуванням імена констант чутливі до регістру. Для кожної константи це можна змінити, вказавши в якості значення аргументу **Нечутливість_до_регістру** значення **True**. Існує угода, за якою імена констант завжди пишуться у верхньому регістрі.

Отримати значення константи можна, вказавши її ім'я. На відміну від змінних, не потрібно випереджати ім'я константи символом **\$**. Крім того, для отримання значення константи можна використовувати функцію **constant()** з ім'ям константи як параметр.

```
<?Php
// визначаємо константу PASSWORD
define("PASSWORD", "qwerty");
// визначаємо регістронезалежну константу PI зі значенням 3.14
define( "PI", "3.14", True);
// виведемо значення константи PASSWORD, тобто qwerty
echo (PASSWORD);
echo "<br>";
// теж виведе qwerty
echo constant("PASSWORD");
echo (password);
/* Виведе password і попередження, оскільки ми ввели до регістру
   константу PASSWORD */
echo pi;
// виведе 3.14, оскільки константа PI регістронезалежна за визначенням
?>
```

Приклад 3.1.4. Константи в PHP

Крім констант, які декларуються користувачем, про які ми тільки що розповіли, в PHP існує ряд констант, обумовлених самим інтерпретатором. Наприклад, константа **__FILE__** зберігає ім'я файлу програми (і шлях до нього), яка виконується в даний момент, **__FUNCTION__** містить ім'я функції, **__CLASS__** - ім'я класу, **PHP_VERSION** - версія інтерпретатора PHP. Повний список зумовлених констант можна отримати, прочитавши керівництво по PHP.

Оператори

Оператори дозволяють виконувати різні дії із змінними, константами і виразами. Ми ще не згадували про те, що такий вислів. Вираз можна визначити як все, що завгодно, що має значення. Змінні і константи - це основні і найбільш прості форми виразів. Існує безліч операцій (і відповідних їм операторів), які можна виробляти з виразами. Розглянемо деякі з них докладніше.

Таблиця 3.1.1. Арифметичні оператори		
Позначення	Назва	Приклад
+	додавання	$\$A + \b
-	віднімання	$\$A - \b
*	множення	$\$A * \b
/	розподіл	$\$A / \b
%	Залишок від ділення	$\$A \% \b

Таблиця 3.1.2. Строкові оператори		
Позначення	Назва	Приклад
. (крапка)	Конкатенація (Додавання рядків)	$SC = \$a.\b (Це рядок, що складається з $\$a$ і $\$b$)

Таблиця 3.1.3. Оператори присвоювання			
Позначення	Назва	Опис	Приклад
=	Присвоєння	Змінної зліва від оператора буде присвоєно значення, отримане в результаті виконання будь-яких операцій або змінної / константи з правої сторони	$\$A = (\$b = 4) + 5;$ ($\$a$ буде дорівнювати 9, $\$b$ буде дорівнювати 4)
+=		Скорочення. Додає до змінної число і потім привласнює їй отримане значення	$\$A += 5;$ (еквівалентно $\$A = \$a + 5;$)
. =		Скорочено позначає комбінацію операцій конкатенації і привласнення (спочатку додається рядок, потім отриманий рядок записується в змінну)	$\$B = "Привіт";$ $\$B. = "Всім";$ (еквівалентно $\$B = \$b."Всім";$) В результаті: $\$B = "Привіт всім"$

Таблиця 3.1.4. Логічні оператори			
Позначення	Назва	Опис	Приклад
and	I	$\$a$ і $\$b$ істинні (True)	$\$A \text{ and } \b
&&	I		$\$A \&\& \b
or	або	Хоча б одна зі змінних $\$a$ або $\$b$ істинна (можливо, що і обидві)	$\$A \text{ or } \b
	або		$\$A \b
xor	Що виключає або	Одна з змінних істинна. Випадок, коли вони обидві істинні, виключається	$\$A \text{ xor } \b
!	Інверсія (NOT)	Якщо $\$A = \text{True}$, то $!\$A = \text{False}$ і навпаки	$!\$a$

Таблиця 3.1.5. Оператори порівняння			
позначення	Назва	опис	приклад
==	рівність	Значення змінних рівні	$\$A == \b
===	еквівалентність	Рівні значення і типи змінних	$\$A === \b
!=	нерівність	Значення змінних не рівні	$\$A! = \B
<>	нерівність		$\$A <> \b
!==	нееквівалентність	Змінні не еквівалентні	$\$A! == \b
<	менше		$\$A < \b
>	більше		$\$A > \b

<=	Менше або дорівнює	\$A <= \$b
>=	Більше або дорівнює	\$A >= \$b

Таблиця 3.1.6. Оператори інкремента і декремента			
позначення	Назва	опис	приклад
<code>++\$a</code>	пре інкремент	Збільшує <code>\$a</code> на одиницю і повертає <code>\$a</code>	<pre><? \$a = 4; echo "Повинно бути 4: ".\$a++; echo "Повинно бути 6: " . ++\$a; ?></pre>
<code>\$a++</code>	пост-інкремент	Повертає <code>\$a</code> , Потім збільшує <code>\$a</code> на одиницю	
<code>--\$a</code>	пре декремент	Зменшує <code>\$a</code> на одиницю і повертає <code>\$a</code>	
<code>\$a--</code>	пост-декремент	Повертає <code>\$a</code> , Потім зменшує <code>\$a</code> на одиницю	

6. Типи даних.

PHP підтримує вісім простих типів даних.

Чотири скалярних типи:

boolean (логічний);

integer (цілий);

float (з плаваючою точкою);

string (строковий).

Два змішаних типи:

array (масив);

object (об'єкт).

І два спеціальних типи:

resource (ресурс);

NULL.

У *PHP* не прийнято явне оголошення типів змінних. Переважно, щоб це робив сам інтерпретатор під час виконання програми в залежності від контексту, в якому використовується змінна. Розглянемо по порядку всі перераховані типи даних.

Тип **boolean** (логічний або логічний тип)

Цей найпростіший тип виражає істинність значення, тобто змінна цього типу може мати тільки два значення – істина **TRUE** або брехня **FALSE**.

Щоб визначити логічний тип, використовують ключове слово **TRUE** або **FALSE**. Обидва регістро-незалежні.

```
<?Php
```

```
$Test = True;
```

```
?>
```

Приклад. логічний тип

Логічні змінні використовуються в різних управляючих конструкціях. Мати логічний тип, тобто приймати тільки два значення, істину або брехня, можуть також і деякі оператори (наприклад, оператор рівності). Вони також використовуються в керуючих конструкціях для перевірки будь-яких умов. Наприклад, в умовній конструкції перевіряється істинність значення оператора або змінної і залежно від результату перевірки

виконуються ті чи інші дії. Тут умова може бути істинно або хибно, що якраз і відображає змінна і оператор логічного типу.

```
<?Php
// Оператор '==' перевіряє рівність і повертає логічне значення
if ($know == False)
    {// якщо $know має значення false
    echo "Вивчай РНР!";
    }
if (!$know)
    {// те ж саме, що і вище, тобто перевірка чи має $know значення false
    echo "Вивчай РНР!";
    }
/* Оператор == перевіряє, чи збігається значення змінної $know з рядком
"Вивчай РНР". Якщо збігається, то повертає true, інакше - false.
Якщо повернуто true, то виконується то, що всередині фігурних дужок */
if ($know == "Вивчай РНР")
    {Echo "Почав вивчати"; }
?>
```

Приклад 3.1.5. Використання логічного типу

Тип integer (цілі).

Цей тип задає число з безлічі цілих чисел $Z = \{..., -2, -1, 0, 1, 2, ...\}$. Цілі можуть бути вказані у десятковій, шістнадцятковій або вісімковій системі числення, за бажанням з попереднім знаком "-" або "+".

Якщо ви використовуєте вісімкову систему числення, ви повинні випередити число 0 (Банкрутом), для використання шістнадцятковій системи потрібно поставити перед числом 0x.

```
<?Php
# Десяткове число
$A = 1234;
# від'ємне число
$A = -123;
# Вісімкове число (еквівалентно 83 в десятковій системі)
$A = 0123;
# Шістнадцяткове число (еквівалентно 26 в десятковій системі)
$A = 0x1A;
?>
```

Розмір цілого залежить від платформи, хоча, як правило, максимальне значення близько двох мільярдів (це 32-бітове знакове). Беззнакові цілі РНР не підтримує.

Якщо ви визначите число, що перевищує межі цілого типу, воно буде інтерпретовано як число з плаваючою крапкою. Також якщо ви використовуєте оператор, результатом роботи якого буде число, що перевищує межі цілого, замість нього буде повернуто число з плаваючою крапкою.

У РНР не існує оператора ділення цілих. Результатом $\frac{1}{2}$ буде число з плаваючою точкою 0.5. Ви можете привести значення до цілого, що завжди округлює його в меншу сторону, або використовувати функцію `round()`. Округлюються значення за стандартними правилами. Для перетворення змінної до конкретного типу потрібно перед змінної вказати в дужках потрібний тип. Наприклад, для перетворення змінної `$A = 0.5` до цілого типу необхідно написати `(Integer) (0.5)` або `(Integer) $a` або використовувати скорочений запис `(Int) (0.5)`. Можливість явного приведення типів за таким принципом існує для всіх типів даних (звичайно, не завжди значення одного типу можна перевести в інший тип). Ми не будемо заглиблюватися в усі тонкощі приведення типів, оскільки РНР робить це автоматично в залежності від контексту.

Тип float (числа з плаваючою точкою).

Числа з плаваючою точкою (Вони ж числа подвійної точності або дійсні числа) можуть бути визначені за допомогою будь-якого з наступних синтаксисів:

```
<?Php
$A = 1.234;
$B = 1.2e3;
$C = 7E-10;
?>
```

Розмір числа з плаваючою точкою залежить від платформи, хоча максимум, як правило, ~ 1.8e308 з точністю близько 14 десяткових цифр.

Тип string (рядки).

Рядок - це набір символів. У PHP символ - це те ж саме, що байт, це означає, що існує рівно 256 різних символів. Це також означає, що PHP не має вбудованої підтримки Unicode. У PHP практично не існує обмежень на розмір рядків, тому немає абсолютно ніяких причин турбуватися про їх довжину.

Рядок у PHP може бути визначений трьома різними способами:

за допомогою одинарних лапок;

за допомогою подвійних лапок;

heredoc-синтаксисом.

Одинарні лапки

Найпростіший спосіб визначити рядок - це укласти її в одинарні лапки "'". Щоб використовувати одинарні лапки всередині рядка, як і в багатьох інших мовах, перед нею необхідно поставити символ зворотної косої межі "\", тобто екранувати її. Якщо зворотна коса риска повинна йти перед одинарною лапкою або бути в кінці рядка, необхідно продублювати її так "\"\"".

Якщо всередині рядка, укладеної в одинарні лапки, зворотний слеш "\" Зустрічається перед будь-яким іншим символом (відмінним від "\"I'"), то він розглядається як звичайний символ і виводиться, як і всі інші. Тому зворотну косу риску необхідно екранувати, тільки якщо вона знаходиться в кінці рядка, перед закриває лапками.

У PHP існує ряд комбінацій символів, що починаються з символу зворотної косої межі. Їх називають керуючими послідовностями, і вони мають спеціальні значення, про які ми розповімо трохи пізніше. Так ось, на відміну від двох інших синтаксисів, змінні і керуючі послідовності для спеціальних символів, що зустрічаються в рядках, взятих в одинарні лапки, не обробляються.

```
<?Php
$expand='sdf';
$either=2323;
echo 'Також ви можете вставляти в рядки символ нового рядка таким чином,
    оскільки це нормально';
// Виведе: Щоб вивести ' треба перед нею поставити \
echo 'Щоб вивести ' треба перед 'Ній поставити \';
// Виведе: Ви хочете видалити C:\*.*?
echo 'Ви хочете видалити C:\\*.*?';
// Виведе: Це не вставить: \n новий рядок
echo 'Це не вставить: \n новий рядок';
// Виведе: Змінні $expand також $either не підставляти
echo 'Змінні $expand також $either'. "Не підставляються ";
?>
```

Приклад 3.1.6. Використання керуючих послідовностей

Подвійні лапки

Якщо рядок укладена в подвійні лапки "", PHP розпізнає більшу кількість керуючих послідовностей для спеціальних символів. Деякі з них наведені в таблиці 3.1.7.

Таблиця 3.1.7. керуючі послідовності	
послідовність	Значення
<code>\n</code>	Новий рядок (LF або 0x0A (10) в ASCII)
<code>\r</code>	Повернення каретки (CR або 0x0D (13) в ASCII)
<code>\t</code>	Горизонтальна табуляція (HT або 0x09 (9) в ASCII)
<code>\\</code>	Зворотна коса риска
<code>\\$</code>	знак долара
<code>\"</code>	Лапки

Повторюємо, якщо ви захочете екранувати будь-який інший символ, зворотна коса риска також буде надрукована!

Найважливішим властивістю рядків у подвійних лапках є обробка змінних.

Here doc

Інший спосіб визначення рядків - це використання heredoc-синтаксису. В цьому випадку рядок повинен починатися з символу `<<<`, Після якого йде ідентифікатор. Закінчується рядок цим же ідентифікатором. Закриває ідентифікатор повинен починатися в першому стовпці рядка. Крім того, ідентифікатор повинен відповідати тим же правилам іменування, що і всі інші мітки в PHP: містити тільки буквено-цифрові символи і знак підкреслення і починатися з цифри або знак підкреслення.

Here doc-текст поводить ся так само, як і рядок в подвійних лапках, при цьому їх не маючи. Це означає, що вам немає необхідності екранувати лапки в heredoc, але ви як і раніше можете використовувати перераховані вище керуючі послідовності. Змінні всередині heredoc теж обробляються.

```
<?Php
```

```
$ Str = <<< EOD
```

Приклад рядка, що охоплює кілька рядків, з використанням heredoc-синтаксису

```
EOD;
```

```
// Тут ідентифікатор - EOD. Нижче ідентифікатор EOD
```

```
$name = 'Вася';
```

```
echo <<< EOD
```

```
Мене звать "$name".
```

```
EOD;
```

```
// це виведе: Мене звать "Вася".
```

```
?>
```

Приклад 3.1.8. Використання heredoc-синтаксису

Зауваження: Підтримка heredoc була додана в PHP 4.

Тип array (масив).

Масив у PHP є впорядкованою картою - тип, який перетворює значення в ключі. Цей тип оптимізований в декількох напрямках, тому ви можете використовувати його як власне масив, список (вектор), хеш-таблицю (що є реалізацією карти), стек, чергу і т.д. Оскільки ви можете мати в якості значення інший масив PHP, можна також легко емулювати дерева.

Визначити масив можна за допомогою конструкції `array()` або безпосередньо задаючи значення його елементів.

Визначення за допомогою array()

```
array(key => value, key1 => value1, ...)
```

Мовна конструкція `array()` приймає в якості параметрів пари **ключ=>значення**, Розділені комами символ `=>` встановлює відповідність між значенням і його ключем. Ключ може бути як цілим числом, так і рядком, а значення може бути будь-якого наявного в PHP

типу. Числовий ключ масиву часто називають індексом. Індування масиву в PHP починається з нуля. Значення елемента масиву можна отримати, вказавши після імені масиву в квадратних дужках ключ шуканого елемента. Якщо ключ масиву представляє собою стандартну запис цілого числа, то він розглядається як число, в іншому випадку - як рядок. Тому запис `$A["1"]` рівнозначна запису `$A[1]`, так само як і `$A["- 1"]` рівнозначна `$A[-1]`.

```
<?Php
$books = array("php" =>"PHP users guide", 12 => true);
echo $books["php"];
// виведе "PHP users guide"
echo $books[12];
// виведе 1
?>
```

Приклад 3.1.9. Масиви в PHP

Якщо для елемента ключ не заданий, то в якості ключа береться максимальний числовий ключ, збільшений на одиницю. Якщо вказати ключ, якому вже було присвоєно якесь значення, то воно буде перезаписано. Починаючи з PHP 4.3.0, якщо максимальний ключ - негативне число, то наступним ключем масиву буде нуль (0).

```
<?Php
// масиви $arr1 $ arr1 еквівалентні
$arr= array (5 => 43, 32, 56, "b" => 12);
$ Arr1 = array (5 => 43, 6 => 32, 7 => 56, "b" => 12);
?>
```

Приклад 3.1.10. Масиви в PHP

Якщо використовувати в якості ключа `TRUE` або `FALSE`, То його значення переводиться відповідно в одиницю і нуль типу `integer`. Якщо використовувати `NULL`, то замість ключа отримуємо порожній рядок. Можна використовувати і саму порожню рядок в якості ключа, при цьому її треба брати в лапки. Так що це не те ж саме, що використання порожніх квадратних дужок. Не можна використовувати в якості ключа масиви і об'єкти.

Визначення за допомогою синтаксису квадратних дужок

Створити масив можна, просто записуючи в нього значення. Як ми вже говорили, значення елемента масиву можна отримати за допомогою квадратних дужок, усередині яких потрібно вказати його ключ, наприклад, `$Book["php"]`. Якщо вказати новий ключ і нове значення, наприклад, `$Book["new_key"] = "new_value"`, То в масив додається новий елемент. Якщо ми не вкажемо ключ, а тільки дамо значення `$Book[] = "new_value"`, То новий елемент масиву буде мати числовий ключ, на одиницю більший максимального існуючого. Якщо масив, в який ми додаємо значення, ще не існує, то він буде створений.

```
<?
$books["key"] = value; // додали в масив $books значення value з ключем key
$books[] = value1; /* Додали в масив значення value1 з ключем 13, оскільки
максимальний ключ у нас був 12 */
?>
```

Для того щоб змінити конкретний елемент масиву, потрібно просто привласнити йому з його ключем нове значення. Змінити ключ елемента не можна, можна тільки видалити елемент (пару ключ / значення) і додати нову. Щоб видалити елемент масиву, потрібно використовувати функцію `unset()`.

```
<?Php
$books = array("php" => "PHP users guide", 12 => true);

$books[] = "Book about Perl"; // додали елемент з ключем (індексом)
// 13 це еквівалентно $books[13] = "Book about Perl";
$books["lisp"] = 123456; /* Це додає до масиву новий
```

```

        елемент з ключем "lisp" і значенням 123456 */
unset($books [12]); // Це видаляє елемент с ключем 12 з масиву
unset($books); // видаляє масив повністю
?>

```

Зауважимо, що, коли використовуються порожні квадратні дужки, максимальний числовий ключ шукається серед ключів, існуючих в масиві з моменту останнього переіндексування. Переіндексувати масив можна за допомогою функції ***array_values()***.

```

<?Php
$arr=
array ( "a", "b", "c"); /* Створюємо масив зі значеннями "A", "b" і "c".
                        оскільки ключі не вказані, вони будуть 0,1,2 відповідно */
print_r($arr); // виводимо масив (і ключі, і значення)
unset($arr[0]);
unset($arr[1]);
unset($arr[2]);
// видаляємо з нього всі значення
print_r($arr); // виводимо масив (і ключі, і значення)
$arr[] = "aa"; // додаємо новий елемент в масив. Його індексом (ключем)
// буде 3, а не 0
print_r($arr);
$arr=array_values($arr); // переіндексіруєм масив
$arr[] = "bb"; // ключем цього елемента буде 1
print_r($arr);
?>

```

Приклад 3.1.11. Переіндексація масиву

Результатом роботи цього скрипта буде:

```

Array ([0] => a [1] => b [2] => c)
Array ()
Array ([3] => aa)
Array ([0] => aa [1] => bb)

```

Тип object (об'єкти).

Об'єкти - тип даних, що прийшов з об'єктно-орієнтованого програмування (ООП). Згідно з принципами ООП, клас - це набір об'єктів, що володіють певними властивостями і методами роботи з ним, а об'єкт відповідно - екземпляр класу. Наприклад, програмісти - це клас людей, які пишуть програми, вивчають комп'ютерну літературу і, крім того, як всі люди, мають ім'я і прізвище. Тепер, якщо взяти одного конкретного програміста, Васю Іванова, то можна сказати, що він є об'єктом класу програмістів, має ті ж властивості, що й інші програмісти, теж має ім'я, пише програми і т.п.

У PHP для доступу до методів об'єкта використовується оператор **>**. Для ініціалізації об'єкту використовується вираз **new**, Що створює в змінній екземпляр об'єкта.

```

<?Php
// створюємо клас людей
class Person
{
// метод, який навчає людину PHP
function know_php ()
{
    echo "Тепер я знаю PHP";
}
}
$Bob = new Person; // створюємо об'єкт
// класу людина

```

```
$Bob -> know_php (); // навчаємо його PHP  
?>
```

Приклад 3.1.12. Об'єкти в PHP

Тип resource (ресурси).

Ресурс- це спеціальна змінна, що містить посилання на зовнішній ресурс (наприклад, з'єднання з базою даних). Ресурси створюються і використовуються спеціальними функціями (наприклад, `mysql_connect()`, `pdf_new()` і т.п.).

Тип Null.

Спеціальне значення NULL говорить про те, що змінна не має значення.

Змінна вважається NULL, якщо:

їй була присвоєна константа NULL (`$Var = NULL`);

їй ще не було присвоєно якесь значення;

вона була видалена за допомогою `unset()`.

Існує тільки одне значення типу NULL – регістро-незалежне ключове слово **NULL**.

Контрольні питання.

1. **Походження PHP:** Хто і коли створив PHP? Як спочатку розшифровувалася абревіатура PHP і як вона розшифровується зараз?
2. **Архітектура:** Як працює PHP на стороні сервера? Поясніть процес обробки запиту клієнта: від відправки HTTP-запиту до отримання згенерованого HTML.
3. **Інтеграція в HTML:** Які існують способи вставки PHP-коду в HTML-документ? Опишіть стандартні теги `<?php ... ?>`, короткі теги `<? ... ?>` та теги у стилі ASP `<% ... %>`.
4. **Змінні:** Як оголошуються змінні в PHP? Який символ обов'язково стоїть перед іменем змінної? Чи потрібно вказувати тип змінної при оголошенні?
5. **Типи даних:** Перелічіть основні типи даних у PHP (boolean, integer, float, string, array, object, resource, NULL). Наведіть приклади кожного з них.
6. **Константи:** Як визначити константу за допомогою функції `define()`? Чим константи відрізняються від змінних (змінність, знак \$, область видимості)?
7. **Рядкові літерали:** У чому різниця між рядками в одинарних (') та подвійних (") лапках? Як працює підстановка змінних у подвійних лапках?
8. **Heredoc-синтаксис:** Що таке heredoc і як його використовувати для створення багаторядкових текстів? Наведіть приклад синтаксису `<<<EOD ... EOD;`.
9. **Оператори виводу:** У чому різниця між конструкціями `echo` та `print`? Яка з них є функцією, а яка — мовною конструкцією?
10. **Арифметичні операції:** Які арифметичні оператори підтримує PHP? Як відбувається автоматичне перетворення типів при математичних обчисленнях?
11. **Логічні оператори:** Опишіть різницю між операторами `and` та `&&`, `or` та `||`. Який пріоритет вони мають?
12. **Перетворення типів:** Як примусово перетворити змінну одного типу в інший (наприклад, рядок у число) за допомогою приведення типів `(int)`, `(string)` тощо?